Security Hardening

Heavy-Duty Electric Vehicle Simulator

STM32H753ZI · Nucleo-H753ZI · Zephyr RTOS 3.7

MCUboot 2.x · ECDSA-P256 · OTA via HTTP

Author mtilocca
Date April 2026
Phases 1 (HTTP Auth) 2 (IWDG) 3a (MCUboot) 3b (OTA)
Status Phases 1–3b complete; Phase 4 (RDP) deferred to production

This document covers the security architecture, implementation decisions, flash layout, key management, and threat model for the embedded simulator.

Contents

Overview

The simulator runs on an STM32H753ZI microcontroller (Cortex-M7, 480 MHz, 2 MB flash, 1 MB RAM) connected via Ethernet to a LAN. Security was added in four independent layers so that no single failure collapses all protections.

Phase	Control	Primary Threat Addressed	Status
1	HTTP Bearer token + session cookie	Unauthenticated HTTP commands	
2	IWDG hardware watchdog	Firmware hang / DoS	
3a	MCUboot ECDSA-P256 signature	Unsigned firmware execution	
3b	Authenticated OTA endpoint	Unauthenticated firmware update	
4	RDP Level 1 option bytes	Flash readback / extraction	Deferred

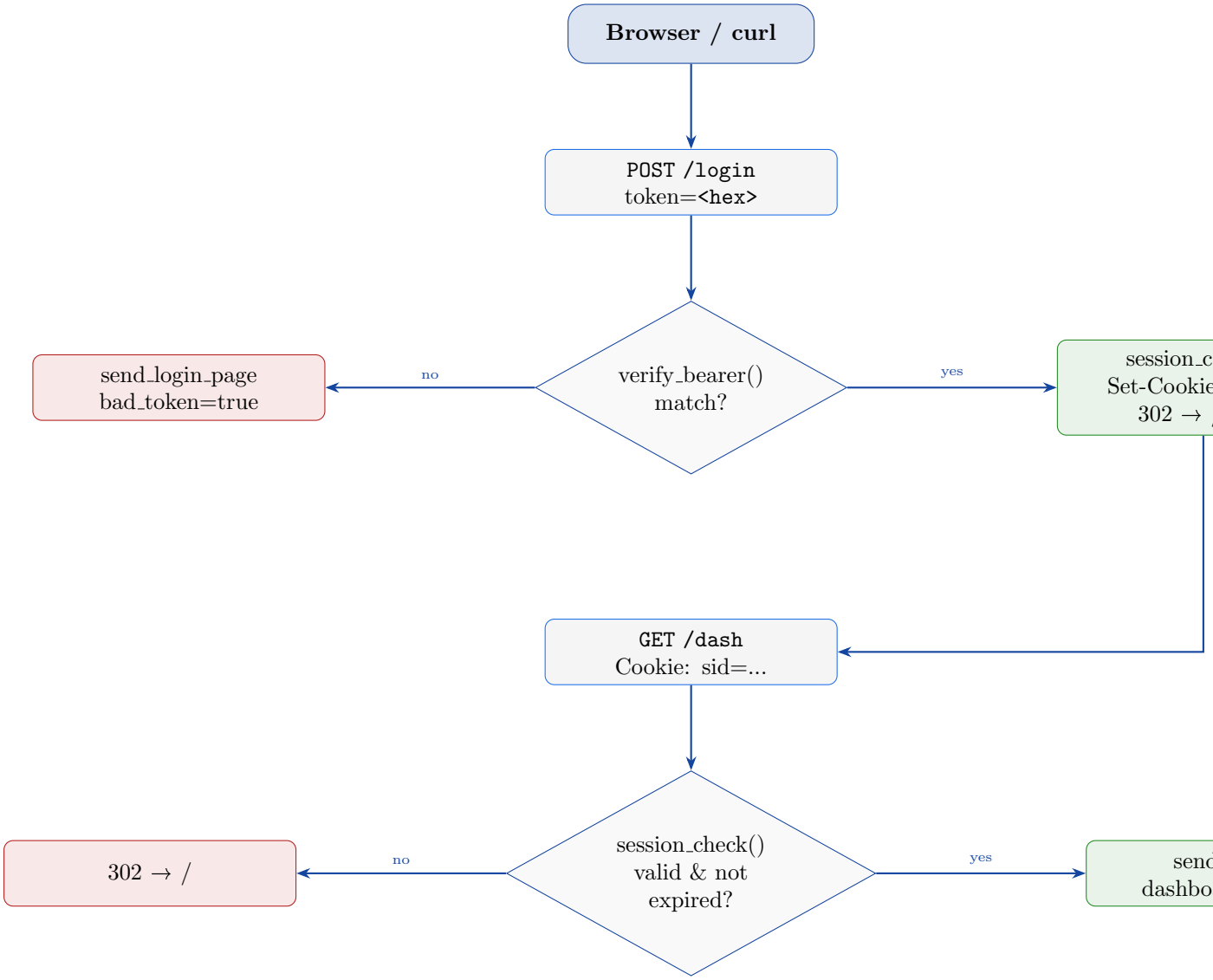
Phase 1 — HTTP Authentication

Token Architecture

A 64-character hexadecimal Bearer token (256 bits of entropy) is generated once at deployment by `scripts/gen_http_token.sh` and stored in `http_auth.hpp` (gitignored). The token never changes between reboots; it is the shared secret between the operator and the board.

Browser sessions are managed separately: a successful `POST /login` with the correct token creates a 32-character random hex session token stored in an on-device table of four slots with a one-hour TTL.

Authentication Flow



Residual Risk

Without TLS, the Bearer token and session cookie are transmitted in plaintext over the LAN. A passive observer on the same subnet can capture and replay the token. This is mitigated in the current deployment by a physically isolated LAN; HTTPS will be added in the next phase to eliminate this residual risk.

Phase 2 — IWDG Hardware Watchdog

Configuration

Parameter	Value
Clock source	LSI (32 kHz, independent of main PLL)
Prescaler (PR)	6 → divide-by-256 → 125 Hz tick
Reload (RLR)	624 → timeout = 625/125 = **5.0 s**
Zephyr API timeout	1 s (`wdt_install_timeout`, `window.max=1000`)
Grace period	10 s (watchdog thread feeds every 500 ms while threads start)
Monitoring	`k_sem_take(g_wdt_sem, K_MSEC(500))` — given by plant_thread

Early Boot Kick Sequence

The IWDG cannot be stopped once started and persists across warm resets. Before the Zephyr thread scheduler starts, the watchdog timer could expire if it was inherited from a previous session. Four `SYS_INIT_NAMED` hooks write `0xAAAA` directly to the IWDG1 key register (hardware kick, no driver needed):

```
static volatile uint32_t* const IWDG1_KR = (volatile uint32_t*)0
    x58004800U;

static int iwdg_early_kick(void) { *IWDG1_KR = 0xAAAAU; return 0; }

SYS_INIT_NAMED(iwdg_kick_pk1, iwdg_early_kick, PRE_KERNEL_1, 0);
SYS_INIT_NAMED(iwdg_kick_pk2, iwdg_early_kick, PRE_KERNEL_2, 0);
SYS_INIT_NAMED(iwdg_kick_pok, iwdg_early_kick, POST_KERNEL,  0);
SYS_INIT_NAMED(iwdg_kick_app, iwdg_early_kick, APPLICATION,  0);
```

Listing 1: Early IWDG kicks in watchdog_thread.cpp

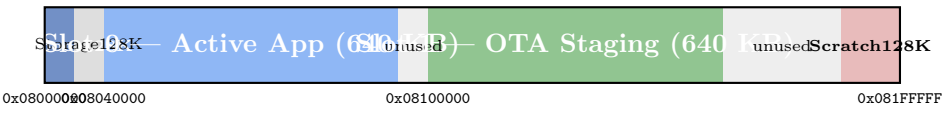
Boot-Loop Root Cause (Postmortem)

During Phase 2 integration the board entered a hard reset loop at 52 ms post-power-on. Root cause: the IWDG was inherited from a prior session with a short timeout and no early kick was issued. The fix above — direct register writes at every SYS_INIT level — ensures the counter is reset before any of the four boot phases can exhaust it. Full postmortem: `docs/zephyr/IWDG_BOOT_LOOP_POSTMORTEM.md`.

Phase 3a — MCUboot Secure Bootloader

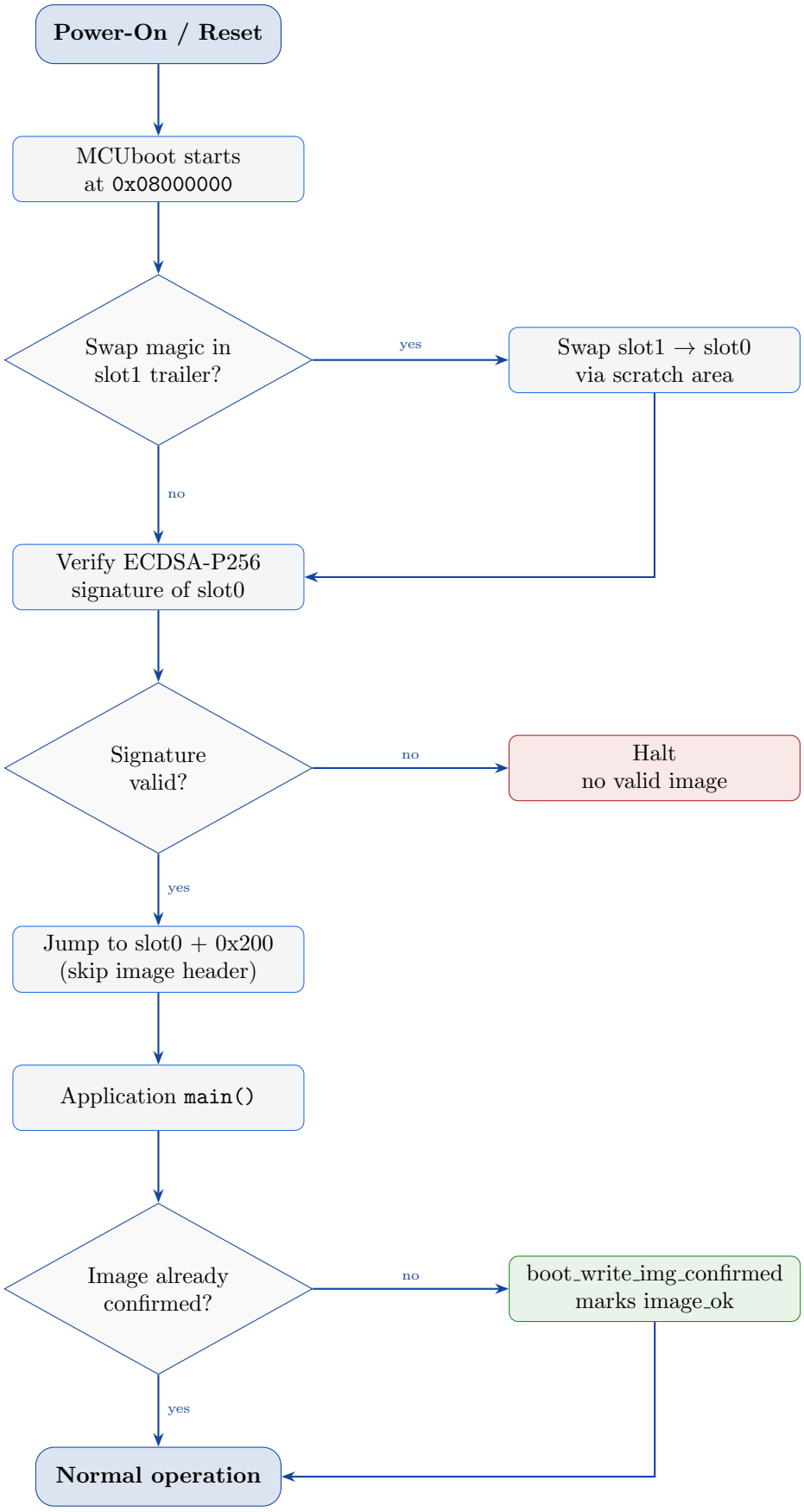
Flash Partition Layout

The STM32H753ZI has 2 MB dual-bank flash with 128 KB sectors. The board default partitions were overridden in both `app.overlay` and `sysbuild/mcuboot.overlay`.



Partition	Address	Size	Sectors	Purpose
`boot_partition`	`0x08000000`	128 KB	Bank1 S0	MCUboot bootloader
`storage_partition`	`0x08020000`	128 KB	Bank1 S1	Board default, unused
`slot0_partition`	`0x08040000`	640 KB	Bank1 S2–6	Active application
(unused)	`0x080E0000`	128 KB	Bank1 S7	Reserved
`slot1_partition`	`0x08100000`	640 KB	Bank2 S0–4	OTA staging
(unused)	`0x081A0000`	256 KB	Bank2 S5–6	Reserved
`scratch_partition`	`0x081E0000`	128 KB	Bank2 S7	MCUboot scratch

Boot Verification Flow



Signing Key Management

Artefact	Location	Notes
Private key (`mcuboot_signing_key.pem`)	`zephyr/`	ECDSA-P256; gitignored; genera
Public key	Embedded in MCUboot binary	Derived from private key at build
Signed app (`zephyr.signed.bin`)	Build output	Signed by `imgtool` during `west`

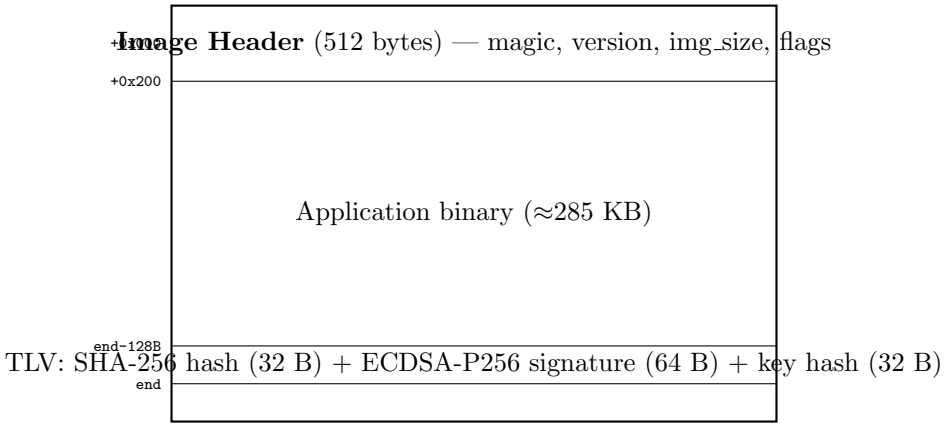
Generate the signing key (run once):

```
python3 /path/to/mcuboot/scripts/imgtool.py keygen \
    -k zephyr/mcuboot_signing_key.pem -t ecdsa-p256
```

The public key is embedded in MCUboot at compile time via `MCUBOOT_SIGNING_KEY_FILE` in `CMakeLists.txt`. The private key must be available at every build that produces a firmware image; it is never stored on the device.

Image Format

Each signed binary has the following structure in flash:



Key DTS Lesson: MCUboot `chosen` Override

The board DTS (`nucleo_h753zi.dts`) sets `zephyr,code-partition = &slot0_partition` in its chosen node. Without an explicit override, MCUboot links itself to slot0 (0x08040000) instead of `boot_partition` (0x08000000) — meaning the app flash writes overwrite the bootloader.

The fix, in `sysbuild/mcuboot.overlay`:

```
/ {
    chosen {
        zephyr,code-partition = &boot_partition;
    };
};
```

Listing 2: MCUboot chosen override

Phase 3b — OTA Firmware Update

API Endpoints

Route	Method	Auth	Purpose
`/ota`	GET	Session or Bearer	Browser upload page with progress bar
`/api/firmware`	POST	Session or Bearer	Stream binary to slot1, arm upgrade
`/api/reboot`	POST	Session or Bearer	Cold reboot into MCUboot swap

Upload Protocol

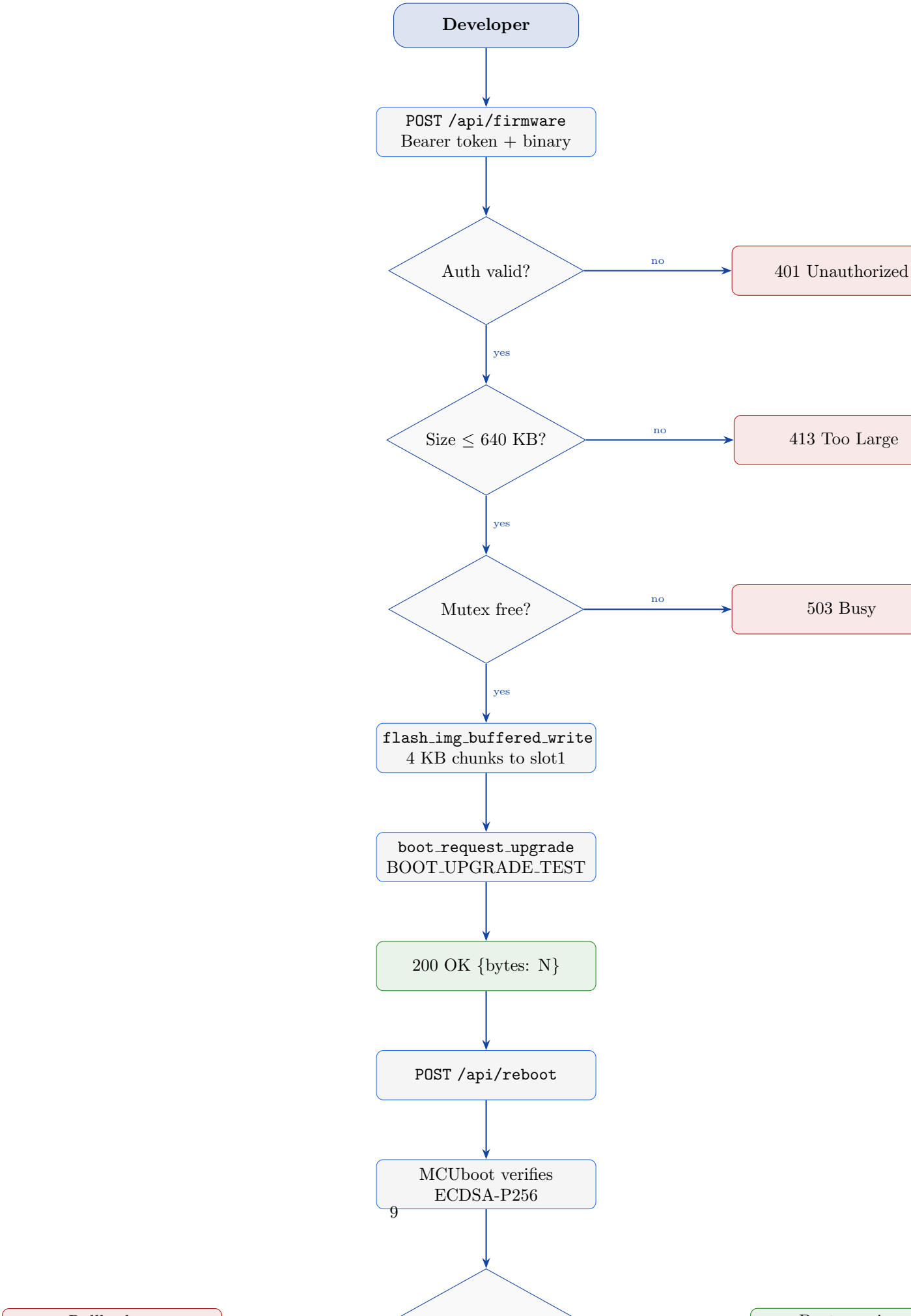
```
TOKEN="<64-char hex from http_auth.hpp>"

# Upload firmware
curl -X POST http://192.168.1.80/api/firmware       \
     -H "Authorization: Bearer $TOKEN"              \
     -H "Content-Type: application/octet-stream"    \
     --data-binary @build/zephyr/zephyr.signed.bin
# Response: {"status":"ok","bytes":292772}

# Trigger swap
curl -X POST http://192.168.1.80/api/reboot         \
     -H "Authorization: Bearer $TOKEN"
# Response: {"status":"rebooting"}
```

Listing 3: OTA via curl

OTA State Machine


Developer
POST /api/firmware
Bearer token + binary
Auth valid?
no
401 Unauthorized
yes
Size ≤ 640 KB?
no
413 Too Large
yes
Mutex free?
no
503 Busy
yes
flash_img_buffered_write
4 KB chunks to slot1
boot_request_upgrade
BOOT_UPGRADE_TEST
200 OK {bytes: N}
POST /api/reboot
MCUboot verifies
ECDSA-P256

Rollback Safety

MCUboot's `BOOT_UPGRADE_TEST` mode ensures the new image must explicitly confirm itself within one boot cycle or MCUboot restores the previous image:

1. Upload sets swap magic in slot1 trailer; `image_ok = 0`.
2. On next boot, MCUboot swaps slot1 → slot0 and boots new image.
3. `boot_write_img_confirmed()` in `main.cpp` writes `image_ok = 1`.
4. On the *next* boot, MCUboot sees `image_ok = 1` and does not swap back.
5. If the new image crashes before reaching `main()`, the next boot finds `image_ok = 0` and MCUboot rolls back automatically.

Implementation Notes

- `CONFIG_IMG_ERASE_PROGRESSIVELY=y`: sectors are erased on the first write to them — no full pre-erase of slot1 required before upload.
- `K_MUTEX_DEFINE(g_ota_mutex)`: prevents concurrent uploads from corrupting the flash write context.
- Static 4 KB receive buffer (`s_rx_buf`): prevents stack overflow on the HTTP server thread during large uploads.
- `handle_api_reboot()` calls `zsock_close(fd)` then waits 200 ms before `sys_reboot(SYS_REBOOT_COLD)` to allow TCP to flush the response.

Threat Model

Threat	Control	Residual Risk
Unauthenticated HTTP command	Bearer token (256-bit)	Token in plaintext on LAN (no TLS yet)
Session hijack	1-hour TTL, 4-slot table	Cookie in plaintext (no TLS yet)
Rogue firmware via OTA	ECDSA-P256 signature required	Attacker needs private key
Bricked device from bad OTA	Test-mode rollback	None — automatic recovery
Firmware extraction via SWD	(Phase 4) RDP Level 1	Currently readable
Firmware hang / DoS	IWDG 1 s timeout → reboot	None for single-fault
Power-loss during OTA swap	MCUboot scratch area + test-mode	None — swap is resumable
Replay of captured token	Bearer token static (no nonce)	Mitigated by HTTPS (planned)

Phase 4 Preview — RDP Level 1

Not yet applied. Will be set as the absolute last production step.

Setting the `RDP` option byte to `0xBB` activates Read Protection Level 1:

- SWD/JTAG can halt the CPU and program flash — development and OTA workflow unchanged.

- SWD/JTAG **cannot read back** flash contents — firmware extraction is blocked.
- The ROM UART bootloader can write new firmware but cannot read back.
- Downgrading to Level 0 triggers a **full mass erase** — all firmware is wiped. After mass erase, the board must be re-flashed from scratch via JTAG or ROM UART bootloader.

After RDP Level 1, OTA (Phase 3b) becomes the primary in-field update mechanism.

Kconfig and File Reference

Key Kconfig Options

Option	Value	Purpose
CONFIG_BOOTLOADER_MCUBOOT	y	App linked to slot0, adds image header
CONFIG_IMG_MANAGER	y	OTA and confirmation APIs
CONFIG_FLASH_MAP	y	Flash area abstraction
CONFIG_STREAM_FLASH	y	Streaming write for OTA
CONFIG_IMG_ERASE_PROGRESSIVELY	y	Erase on-demand during write
CONFIG_WATCHDOG	y	Zephyr WDT driver
CONFIG_BOOT_SIGNATURE_TYPE_ECDSA_P256	y	MCUboot signature algorithm
CONFIG_BOOT_MAX_IMG_SECTORS	512	Max sectors per slot

Critical Files

File	Purpose
zephyr/app.overlay	Flash partition layout for app
zephyr/sysbuild/mcuboot.overlay	MCUboot partition layout + chosen override
zephyr/sysbuild/mcuboot.conf	MCUboot Kconfig (ECDSA-P256)
zephyr/sysbuild.conf	SB_CONFIG_BOOTLOADER_MCUBOOT=y
zephyr/CMakeLists.txt	MCUBOOT_SIGNING_KEY_FILE
zephyr/mcuboot_signing_key.pem	ECDSA-P256 private key (gitignored)
zephyr/src/main.cpp	boot_write_img_confirmed()
zephyr/src/http/http_ota.cpp	OTA upload, page, reboot handlers
zephyr/src/http/http_server.cpp	Route table (includes OTA routes)
zephyr/src/watchdog/watchdog_thread.cpp	IWDG kicks + semaphore monitoring